

Altino Ávila
Guilherme Diniz
Samuel
Jessiê

Lixeira Automática utilizando Arduino Uno

São Paulo
2026

Altino Ávila
Guilherme Diniz
Samuel
Jessiê

Lixeira Automática utilizando Arduino Uno

Tecnologia Frugal

Centro Universitário Senac
Análise e Desenvolvimento de Sistemas
Disciplina: Conceitos de computação

Orientador: Fábio Brussolo

São Paulo
2026

Sumário

1	INTRODUÇÃO	3
1.1	Contextualização do Projeto	3
1.2	O Conceito de Tecnologia Frugal	3
1.3	Objetivos do Projeto	3
2	FUNDAMENTAÇÃO TEÓRICA	4
2.1	Arduino Uno	4
2.2	Sensor Ultrassônico HC-SR04	4
2.3	Servo Motor SG90	4
3	ESPECIFICAÇÕES DO SISTEMA	5
3.1	Requisitos Funcionais	5
3.2	Requisitos Não-Funcionais	5
3.3	Lista de Componentes	5
3.4	Diagrama de Conexões Eletrônicas	5
4	IMPLEMENTAÇÃO	7
4.1	Entendendo a Lógica	7
4.2	Código Arduino Comentado	7
4.3	Fluxo de Funcionamento	12
4.4	Processo de Montagem	13
5	MELHORIAS PROPOSTAS	14
5.1	Melhorias Imediatas	14
5.2	Melhorias de Longo Prazo (Integração IoT)	14
5.3	Impacto das Melhorias	14
6	CONCLUSÃO	15
6.1	Resumo do Projeto	15
6.2	Aprendizados Técnicos	15
6.3	Pitch	15

1 Introdução

1.1 Contextualização do Projeto

Link do projeto que utilizamos como base:

<<https://www.youtube.com/watch?v=h7AmHDMgpJY>>

O presente projeto propõe o desenvolvimento de uma lixeira automática baseada em Arduino Uno, utilizando sensor ultrassônico HC-SR04 para detecção de proximidade e servo motor SG90 para acionamento automático da tampa.

Este protótipo representa a

Fase 1 (automação local). Toda a estrutura de sensores e a lógica de programação foram desenvolvidas com o intuito de preparar o terreno para a **Fase 2**. Essa futura fase incluirá um módulo com conectividade Wi-Fi (como o ESP32) para enviar dados para a internet, avisando no celular, por exemplo, quando a lixeira estiver cheia.

1.2 O Conceito de Tecnologia Frugal

O nosso projeto adota o preceito da **Tecnologia Frugal**. Em termos simples, isso significa *fazer mais com menos*. É uma abordagem voltada para criar soluções eficientes, acessíveis e de baixo custo, sem perder a funcionalidade.

Em vez de utilizarmos estruturas caras de metal impresso em 3D ou plásticos sob medida, aplicamos o conceito frugal utilizando papelão (material reciclável e de fácil acesso) para a montagem física da lixeira, combinado com eletrônica de entrada. O objetivo é provar que é possível criar automação inteligente gastando muito pouco.

1.3 Objetivos do Projeto

Os objetivos deste projeto são:

1. Desenvolver uma lixeira automática funcional utilizando Arduino Uno;
2. Implementar abertura automática sem contato físico;
3. Utilizar sensores ultrassônicos para detecção de presença;
4. Controlar a tampa utilizando servo motor SG90;
5. Implementar fechamento automático temporizado;
6. Criar uma base para futuras integrações IoT;

2 Fundamentação Teórica

2.1 Arduino Uno

O Arduino Uno é uma plataforma de prototipagem eletrônica baseada no microcontrolador ATmega328P. A placa possui 14 portas digitais, 6 entradas analógicas e clock de 16 MHz. Funciona como o *cérebro* do nosso projeto, coordenando os sensores e os motores.

2.2 Sensor Ultrassônico HC-SR04

O HC-SR04 é um sensor capaz de medir distâncias utilizando ondas ultrassônicas de 40 kHz.

Como explicar de forma simples: Ele funciona exatamente como a ecolocalização de um morcego ou o sonar de um submarino. O sensor *grita* um som muito agudo (que não conseguimos ouvir) e conta quanto tempo o eco demora para bater na mão de uma pessoa e voltar. Sabendo o tempo da viagem do som, o Arduino calcula a distância exata.

O módulo possui quatro pinos:

- VCC (Energia);
- GND (Terra);
- Trigger (O *alto-falante* que envia o pulso sonoro);
- Echo (O *microfone* que escuta o retorno).

2.3 Servo Motor SG90

O SG90 é um micro servo motor utilizado para controle angular. Seu funcionamento ocorre através de sinais enviados pelo Arduino.

Como explicar de forma simples: Diferente do motor de um carrinho de controle remoto que fica girando sem parar, o servo motor funciona mais como a articulação de um *braço robótico*. Ele sabe exatamente em qual ângulo está posicionado e consegue mover a tampa da lixeira controladamente do ponto A (fechada) ao ponto B (aberta), e parar lá.

3 Especificações do Sistema

3.1 Requisitos Funcionais

1. Detectar presença em até 20 cm;
2. Abrir automaticamente a tampa;
3. Fechar automaticamente após 5 segundos;
4. Registrar eventos via Serial Monitor.

3.2 Requisitos Não-Funcionais

- Baixo consumo energético;
- Tempo de resposta inferior a 1 segundo;
- Fácil manutenção;
- Baixo custo de implementação;
- Funcionamento estável em ambientes internos.

3.3 Lista de Componentes

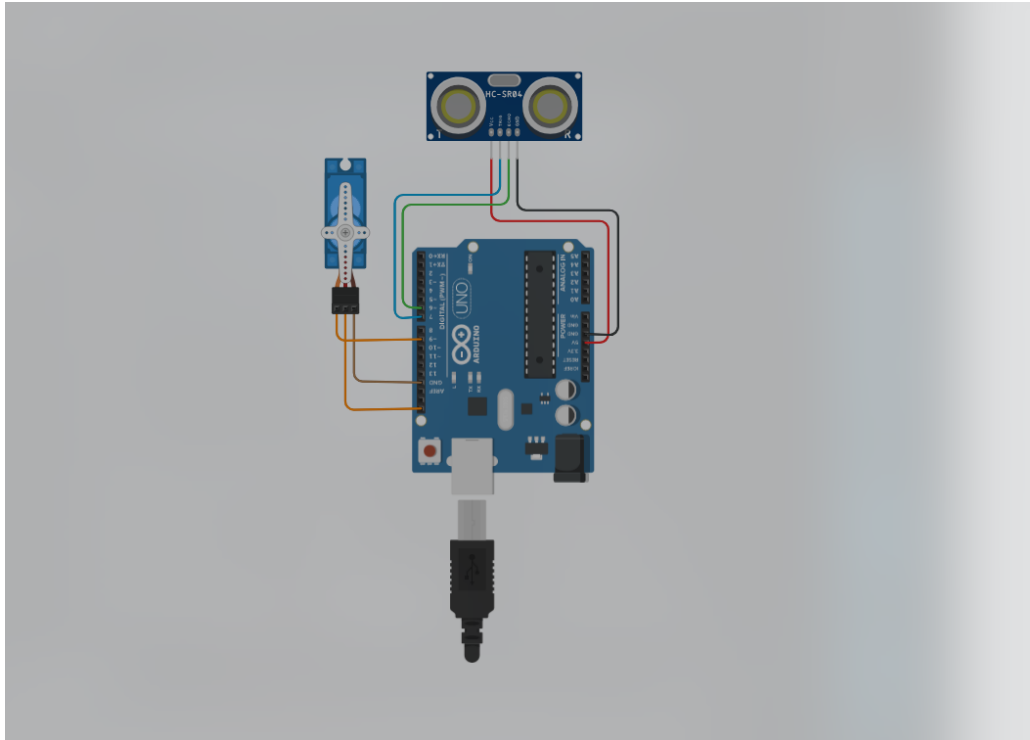
Tabela 1 – Lista de Componentes

Componente	Quantidade	Custo Médio (R\$)
Arduino Uno	1	35,00
HC-SR04	1	19,00
Servo SG90	1	20,00
Kit Jumpers	20	28,00

3.4 Diagrama de Conexões Eletrônicas

O sistema é composto por um sensor ultrassônico conectado ao Arduino Uno, responsáveis pela detecção de presença. O servo motor recebe os dados da distância medida e abre/fecha a tampa.

Figura 1 – Diagrama de conexões elétricas e montagem do sistema



Fonte: Adaptado dos autores (2026).

Tabela 2 – Mapeamento detalhado das conexões do circuito

Componente	Pino do Componente	Conexão no Arduino	Cor do Fio
Sensor HC-SR04	VCC	5V	Vermelho
	GND	GND	Preto
	TRIG	Digital 7	Azul
	ECHO	Digital 6	Verde
Servo Motor SG90	VCC	5V	Vermelho
	GND	GND	Marrom / Preto
	PWM (Sinal)	Digital 9	Laranja / Amarelo

Fonte: Os autores (2026).

4 Implementação

4.1 Entendendo a Lógica

A maior preocupação ao programar algo mecânico (que se move) é evitar que o sistema tente fazer duas coisas ao mesmo tempo ou que trave. Para resolver isso, usamos um conceito na programação chamado **Máquina de Estados**.

Em vez de usar comandos que pausam o sistema (como o `delay()`), o código funciona como um semáforo ou um árbitro de jogo. A lixeira tem três **estados** definidos:

1. **FECHADO:** A lixeira está parada, apenas prestando atenção no sensor de proximidade.
2. **ABERTO:** A tampa abre e começa a contar um tempo interno. Ela não fechará até ter certeza absoluta de que a mão da pessoa já se afastou do sensor.
3. **COOLDOWN (Descanso):** Após fechar, a lixeira entra numa rápida pausa de segurança. Isso serve para que ela não reaja acidentalmente ao movimento da própria tampa se fechando ou a alguém passando rapidamente em frente a ela.

Essa separação garante um funcionamento limpo e profissional.

4.2 Código Arduino Comentado

Código no GitHub: <<https://github.com/sunnycidal/ardulixoino>>

```
1
2 #include <Servo.h> // Biblioteca para controlar o motor
3
4 // =====
5 // CONFIGURACOES
6 // =====
7 #define PINO_SERVO          9
8 #define PINO_TRIG           7
9 #define PINO_ECHO           6
10
11 #define ANGULO_FECHADO      105
12 #define ANGULO_ABERTO       230
13 #define VELOCIDADE_ABRIR    5
14 #define VELOCIDADE_FECHAR   3
15
```

```

16 #define DISTANCIA_CM                20
17
18 #define TEMPO_MINIMO_ABERTO         3500
19 #define COOLDOWN_FECHAMENTO         2000
20 #define TEMPO_MAO_AUSENTE           3000
21
22 // =====
23 //  ESTADOS
24 // =====
25 enum Estado { FECHADO, ABERTO, COOLDOWN };
26 Estado estado = FECHADO;
27
28 unsigned long marcadorTempoAberto = 0;
29 unsigned long marcadorMaoAusente   = 0;
30 unsigned long marcadorCooldown     = 0;
31 unsigned long ultimoLog             = 0;
32 #define INTERVALO_LOG 500
33
34 // =====
35 //  SERVO
36 // =====
37 Servo servo;
38 int posicaoAtual = ANGULO_FECHADO;
39
40 void moverServoPara(int destino, int velocidade) {
41     int passo = (destino > posicaoAtual) ? 1 : -1;
42     while (posicaoAtual != destino) {
43         posicaoAtual += passo;
44         servo.write(posicaoAtual);
45         delay(velocidade);
46     }
47 }
48
49 // =====
50 //  SENSOR
51 // =====
52 int distanciaAtual = 0;
53
54 int medirDistancia() {
55     digitalWrite(PINO_TRIG, LOW);
56     delayMicroseconds(2);
57     digitalWrite(PINO_TRIG, HIGH);

```

```

58     delayMicroseconds(10);
59     digitalWrite(PINO_TRIG, LOW);
60     long duracao = pulseIn(PINO_ECHO, HIGH, 30000);
61     return duracao * 0.034 / 2;
62 }
63
64 bool maoDetectada() {
65     distanciaAtual = medirDistancia();
66     return (distanciaAtual > 0 && distanciaAtual <= DISTANCIA_CM);
67 }
68
69 bool maoConfirmedamenteAusente() {
70     if (maoDetectada()) {
71         marcadorMaoAusente = millis();
72         return false;
73     }
74     return (millis() - marcadorMaoAusente >= TEMPO_MAO_AUSENTE);
75 }
76
77 // =====
78 //  LOG (MONITOR SERIAL)
79 // =====
80 String barraProgresso(float percentual, int tamanho = 10) {
81     int preenchido = (int)(constrain(percentual, 0.0, 1.0) *
82         tamanho);
83     String barra = "[";
84     for (int i = 0; i < tamanho; i++)
85         barra += (i < preenchido) ? "#" : "-";
86     barra += "]";
87     return barra;
88 }
89
90 void logTransicao(String msg) {
91     Serial.println();
92     Serial.println(">> " + msg);
93 }
94
95 void logStatus() {
96     if (millis() - ultimoLog < INTERVALO_LOG) return;
97     ultimoLog = millis();
98
99     String barSensor = barraProgresso(1.0 - (distanciaAtual /

```

```

    (float)DISTANCIA_CM));
99  Serial.print("Sensor: ");
100 Serial.print(barSensor);
101 Serial.print("  ");
102 Serial.print(distanciaAtual);
103 Serial.print("cm");
104
105 switch (estado) {
106     case FECHADO:
107         Serial.print("  | FECHADA  |  Aguardando mao...");
108         break;
109
110     case ABERTO: {
111         unsigned long tempoAberto  = millis() -
            marcadorTempoAberto;
112         unsigned long tempoRestante = 0;
113         if (TEMPO_MINIMO_ABERTO > tempoAberto)
114             tempoRestante = TEMPO_MINIMO_ABERTO - tempoAberto;
115
116         Serial.print("  |  ABERTA  |  ");
117         if (tempoRestante > 0) {
118             float prog = tempoAberto / (float)TEMPO_MINIMO_ABERTO;
119             Serial.print("Minimo: ");
120             Serial.print(barraProgresso(prog));
121             Serial.print("  ");
122             Serial.print(tempoRestante / 1000.0, 1);
123             Serial.print("s");
124         } else if (!maoConfirmedamenteAusente()) {
125             unsigned long ausente = millis() - marcadorMaoAusente;
126             Serial.print("Aguard. mao sair: ");
127             Serial.print(barraProgresso(ausente /
                (float)TEMPO_MAO_AUSENTE));
128         } else {
129             Serial.print("Fechando em instantes...");
130         }
131         break;
132     }
133
134     case COOLDOWN: {
135         unsigned long restante = COOLDOWN_FECHAMENTO - (millis() -
            marcadorCooldown);
136         float prog = 1.0 - restante / (float)COOLDOWN_FECHAMENTO;

```

```

137     Serial.print(" | FECHADA | Cooldown: ");
138     Serial.print(barraProgresso(prog));
139     Serial.print(" ");
140     Serial.print(restante / 1000.0, 1);
141     Serial.print("s");
142     break;
143 }
144 }
145 Serial.println();
146 }
147
148 // =====
149 //  MAQUINA DE ESTADOS
150 // =====
151 void loop() {
152     logStatus();
153
154     switch (estado) {
155
156         case FECHADO:
157             if (maoDetectada()) {
158                 logTransicao("Mao detectada! Abrindo...");
159                 moverServoPara(ANGULO_ABERTO, VELOCIDADE_ABRIR);
160                 marcadorTempoAberto = millis();
161                 marcadorMaoAusente = millis();
162                 estado = ABERTO;
163             }
164             break;
165
166         case ABERTO: {
167             bool minimoDecorrido = (millis() - marcadorTempoAberto >=
168                                     TEMPO_MINIMO_ABERTO);
169             bool maoFora          = maoConfirmedamenteAusente();
170
171             if (minimoDecorrido && maoFora) {
172                 logTransicao("Mao ausente. Fechando...");
173                 moverServoPara(ANGULO_FECHADO, VELOCIDADE_FECHAR);
174                 marcadorCooldown = millis();
175                 estado = COOLDOWN;
176             }
177             break;
178         }
179     }
180 }

```

```

178
179     case COOLDOWN:
180         if (millis() - marcadorCooldown >= COOLDOWN_FECHAMENTO) {
181             logTransicao("Pronta para uso!");
182             estado = FECHADO;
183         }
184         break;
185     }
186
187     delay(200);
188 }
189
190 // =====
191 //  SETUP
192 // =====
193 void setup() {
194     Serial.begin(9600);
195
196     pinMode(PINO_TRIG, OUTPUT);
197     pinMode(PINO_ECHO, INPUT);
198
199     servo.attach(PINO_SERVO);
200     servo.write(ANGULO_FECHADO);
201     delay(1000);
202
203     Serial.println("== Lixeira Inteligente ==");
204     Serial.println();
205 }

```

Listing 4.1 – Código Arduino da lixeira automática

4.3 Fluxo de Funcionamento

1. O sistema inicializa o Arduino;
2. Os sensores começam a realizar leituras;
3. O HC-SR04 detecta aproximação;
4. O Arduino processa os dados;
5. O servo motor abre a tampa;
6. O sistema aguarda 5 segundos;

7. O HC-SR04 confirma que a mão se afastou;
8. A tampa é fechada automaticamente e aguarda recarga.

4.4 Processo de Montagem

1. Separar todos os componentes;
2. Montar o hardware de papelão;
3. Conectar o HC-SR04 ao Arduino;
4. Conectar o servo motor ao arduino;
5. Realizar upload do código;
6. Testar funcionamento;
7. Instalar o circuito dentro da lixeira.

5 Melhorias Propostas

5.1 Melhorias Imediatas

1. Adição de buzzer sonoro;
2. Implementação de sensor para medir a capacidade;
3. Material diferente para a estrutura (ex.: acrílico).

5.2 Melhorias de Longo Prazo (Integração IoT)

1. Integração com módulo ESP32 para conectividade à internet;
2. Desenvolvimento de aplicativo móvel para receber notificações;
3. Monitoramento remoto via Wi-Fi;
4. Uso de energia solar para recarga de bateria interna.

5.3 Impacto das Melhorias

As melhorias propostas podem transformar o projeto em um dispositivo IoT completo, com monitoramento em tempo real e maior interatividade.

6 Conclusão

6.1 Resumo do Projeto

O projeto demonstrou a viabilidade de desenvolvimento de um dispositivo inteligente utilizando componentes de baixo custo e materiais acessíveis (tecnologia frugal).

O protótipo funciona? **Sim.** O sistema é capaz de detectar a aproximação através do sensor ultrassônico, processar a informação via Arduino e acionar o servo motor de forma precisa e sem travamentos, comprovando que a lógica da Máquina de Estados aplicada foi bem-sucedida. O uso do papelão suportou bem a estrutura do motor, validando o conceito de tecnologia frugal.

6.2 Aprendizados Técnicos

Durante o projeto foram desenvolvidos conhecimentos em:

- Lógica de programação focada em Máquina de Estados;
- Sistemas embarcados com Arduino;
- Princípios de ecolocalização com sensores ultrassônicos;
- Conceitos de IoT (preparação de hardware para comunicação remota).

6.3 Pitch

Falta desenvolver